

系统开发工具基础

第 4 周课上实验报告

课程名称： 系统开发工具基础

授课主题： 代码质量；元编程；大杂烩；
课堂综合测试

授课日期： 2026 年 9 月 14 日（周一，第 5—8 节）

姓 名： 张鑫硕

学 号： 25020007170

GitHub: <https://github.com/ADayxs/SystemDevelopmentTools.git>

September 9, 2026

Contents

1	实验概览	3
2	Q13: 建立可执行的本地质量门禁	3
2.1	题目要求	3
2.2	实现步骤	3
2.2.1	1. 复制项目并添加配置	3
2.2.2	2. 补充测试用例	4
2.2.3	3. 运行质量检查并修复	4
2.2.4	4. 编写 <code>check.sh</code>	4
2.3	截图记录	5
2.4	GitHub Commit 记录	5
3	Q14: 让 Make 只重建真正受影响的产物	6
3.1	题目要求	6
3.2	实现步骤	6
3.2.1	1. 创建源文件	6
3.2.2	2. 编写 Makefile	7
3.2.3	3. 验证 Make 行为	7
3.3	截图记录	7
3.4	GitHub Commit 记录	8
4	Q15: 把本地 API 数据转换为可读报告	8
4.1	题目要求	8
4.2	实现步骤	8
4.2.1	1. 创建 <code>packages.json</code>	8
4.2.2	2. 启动本地 HTTP 服务	9
4.2.3	3. 使用 <code>curl</code> 获取数据	9
4.2.4	4. 使用 <code>jq</code> 筛选与排序	9
4.2.5	5. 编写 <code>api_report.sh</code>	9
4.3	截图记录	10
4.4	GitHub Commit 记录	10
5	Q16: 修复并交付一个陌生的小型工具仓库	11
5.1	题目要求	11
5.2	实现步骤	11
5.2.1	1. 复制项目并初始化 Git	11
5.2.2	2. 临时修改导致测试失败	11
5.2.3	3. 定位并修复	11
5.2.4	4. 编写最小 Makefile	12
5.2.5	5. 构建与验证	12
5.2.6	6. Git 提交	12
5.3	截图记录	12
5.4	GitHub Commit 记录	12

6 实验总结

13

1 实验概览

本次实验共 4 道题目，涵盖以下主题：

题号	主题	内容概要	用时
Q13	代码质量	建立可执行的本地质量门禁（ruff + pytest）	12–15 min
Q14	元编程：构建系统	让 Make 只重建真正受影响的产物	12–15 min
Q15	大杂烩：API、jq、CLI	把本地 API 数据转换为可读报告	12–15 min
Q16	课堂综合测试	修复并交付一个陌生的小型工具仓库	12–15 min

2 Q13：建立可执行的本地质量门禁

2.1 题目要求

复用第 10 题完成的 greetlab 包，在 q13 中建立一个轻量本地质量检查。

2.2 实现步骤

2.2.1 1. 复制项目并添加配置

将 q10 复制为 q13，在 pyproject.toml 中加入 ruff 和 pytest 的最小配置：

Listing 1: pyproject.toml —ruff + pytest 配置

```
[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"

[project]
name = "greetlab-25020007170"
version = "0.1.0"

[project.scripts]
sdt-greet = "greetlab.cli:main"

[tool.ruff]
line-length = 88

[tool.ruff.lint]
select = ["E", "F", "W", "I"]

[tool.pytest.ini_options]
testpaths = ["tests"]
```

2.2.2 2. 补充测试用例

新增正常姓名测试和空白姓名测试，每个测试包含有效断言：

Listing 2: tests/test_normal_name.py

```
import subprocess, sys

def test_normal_name_greeting():
    result = subprocess.run(
        [sys.executable, "-m", "greetlab.cli", "--name", "Alice"],
        capture_output=True, text=True,
    )
    assert result.returncode == 0
    assert result.stdout.strip() == "Hello, Alice!"
```

Listing 3: tests/test_blank_name.py

```
import subprocess, sys

def test_blank_name_exits_with_error():
    result = subprocess.run(
        [sys.executable, "-m", "greetlab.cli", "--name", " "],
        capture_output=True, text=True,
    )
    assert result.returncode == 2
    assert result.stdout.strip() == ""
```

2.2.3 3. 运行质量检查并修复

依次运行 ruff format、ruff check 和 pytest，修复全部问题，不得全局忽略规则：

Listing 4: 质量检查流程

```
ruff format .
ruff check .
pytest tests/ -v
```

2.2.4 4. 编写 check.sh

编写 check.sh，依次执行 ruff format --check、ruff check 和 pytest：

Listing 5: check.sh

```
#!/usr/bin/env bash
set -euo pipefail

echo "=== ruff format --check ==="
ruff format --check .

echo "=== ruff check ==="
```

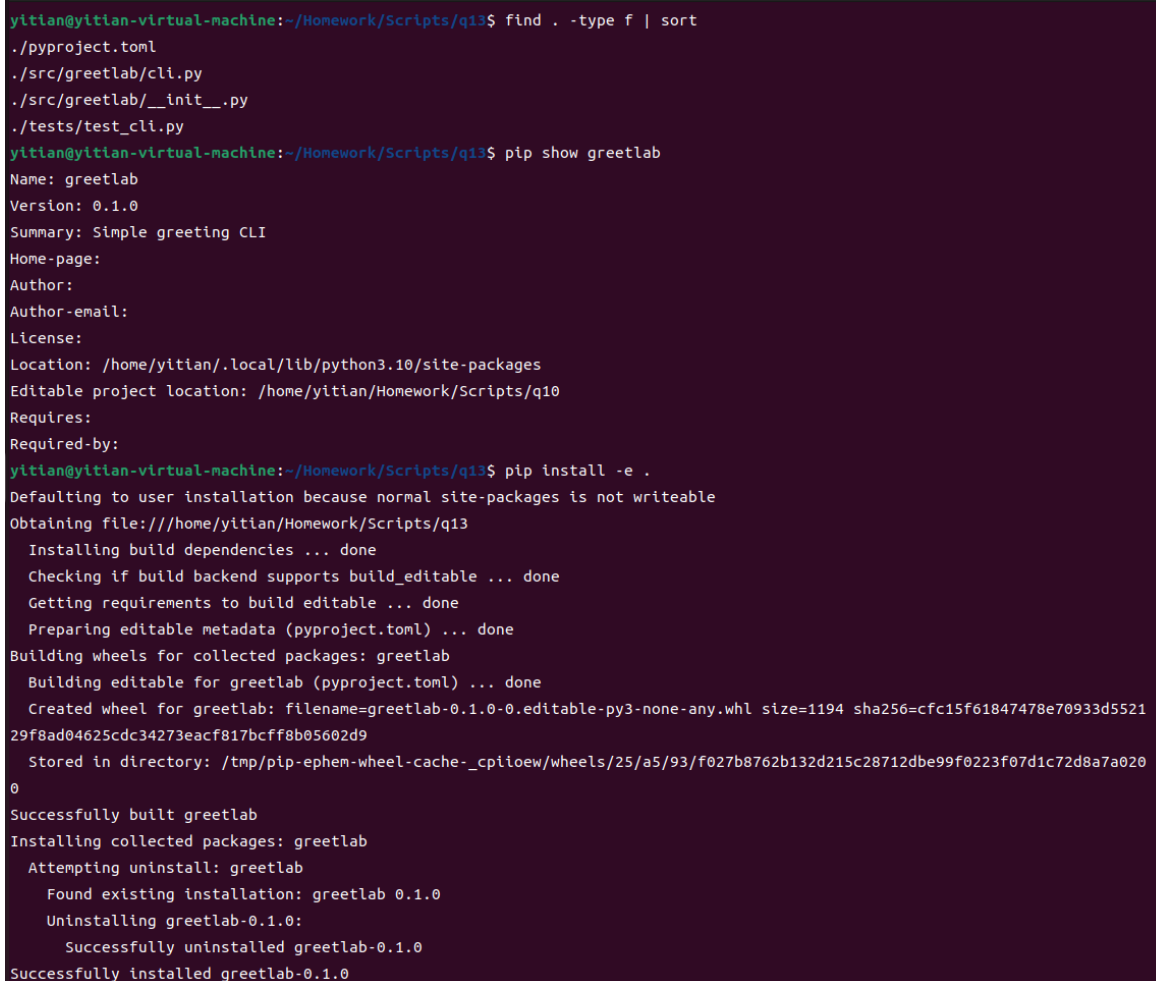
```
ruff check .

echo "=== pytest ==="
pytest tests/ -v

echo "All checks passed."
```

运行 `bash check.sh` 返回 0，全部检查通过。

2.3 截图记录



```
yitian@yitian-virtual-machine:~/Homework/Scripts/q13$ find . -type f | sort
./pyproject.toml
./src/greetlab/cli.py
./src/greetlab/__init__.py
./tests/test_cli.py
yitian@yitian-virtual-machine:~/Homework/Scripts/q13$ pip show greetlab
Name: greetlab
Version: 0.1.0
Summary: Simple greeting CLI
Home-page:
Author:
Author-email:
License:
Location: /home/yitian/.local/lib/python3.10/site-packages
Editable project location: /home/yitian/Homework/Scripts/q10
Requires:
Required-by:
yitian@yitian-virtual-machine:~/Homework/Scripts/q13$ pip install -e .
Defaulting to user installation because normal site-packages is not writeable
Obtaining file:///home/yitian/Homework/Scripts/q13
Installing build dependencies ... done
Checking if build backend supports build_editable ... done
Getting requirements to build editable ... done
Preparing editable metadata (pyproject.toml) ... done
Building wheels for collected packages: greetlab
Building editable for greetlab (pyproject.toml) ... done
Created wheel for greetlab: filename=greetlab-0.1.0-0.editable-py3-none-any.whl size=1194 sha256=cfc15f61847478e70933d5521
29f8ad04625cdc34273eacf817bcff8b05602d9
Stored in directory: /tmp/pip-ephem-wheel-cache-_cpioew/wheels/25/a5/93/f027b8762b132d215c28712dbe99f0223f07d1c72d8a7a020
0
Successfully built greetlab
Installing collected packages: greetlab
Attempting uninstall: greetlab
Found existing installation: greetlab 0.1.0
Uninstalling greetlab-0.1.0:
Successfully uninstalled greetlab-0.1.0
Successfully installed greetlab-0.1.0
```

Figure 1: Q13 —ruff format & ruff check 运行截图

2.4 GitHub Commit 记录

- **Commit 信息:**
feat(q13): add local quality gate with ruff and pytest
- **Commit 截图:** 见附录 6 (图 8)。

```
yitian@yitian-virtual-machine:~/Homework/Scripts/q13$ ruff format .
2 files reformatted, 1 file left unchanged
yitian@yitian-virtual-machine:~/Homework/Scripts/q13$ ruff check .
All checks passed!
yitian@yitian-virtual-machine:~/Homework/Scripts/q13$ pytest
===== test session starts =====
platform linux -- Python 3.10.12, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/yitian/Homework/Scripts/q13
configfile: pyproject.toml
testpaths: tests
collected 3 items

tests/test_cli.py ... [100%]

===== 3 passed in 0.06s =====
yitian@yitian-virtual-machine:~/Homework/Scripts/q13$ nano check.sh
yitian@yitian-virtual-machine:~/Homework/Scripts/q13$ chmod +x check.sh
yitian@yitian-virtual-machine:~/Homework/Scripts/q13$ ./check.sh
=== 1. Checking code format ===
3 files already formatted
=== 2. Checking code style ===
All checks passed!
=== 3. Running tests ===
===== test session starts =====
platform linux -- Python 3.10.12, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/yitian/Homework/Scripts/q13
configfile: pyproject.toml
testpaths: tests
collected 3 items

tests/test_cli.py ... [100%]

===== 3 passed in 0.05s =====
✓ All checks passed!
yitian@yitian-virtual-machine:~/Homework/Scripts/q13$
```

Figure 2: Q13 —pytest 运行与 check.sh 验证截图

3 Q14: 让 Make 只重建真正受影响的产物

3.1 题目要求

在 q14 中按给定内容创建三个源文件,编写 Makefile,包含 all、stats.txt、report.txt 和 clean 目标。

3.2 实现步骤

3.2.1 1. 创建源文件

Listing 6: data.csv

```
name,value
alpha,2
beta,3
gamma,5
```

Listing 7: stats.py

```
import csv

with open("data.csv") as f:
    total = sum(int(r["value"]) for r in csv.DictReader(f))

open("stats.txt", "w").write(str(total))
```

Listing 8: build_report.py

```
text = open("report.md").read()
total = open("stats.txt").read()
open("report.txt", "w").write(f"{text}\nTotal: {total}\n")
```

Listing 9: report.md

```
# Data Report
```

3.2.2 2. 编写 Makefile

Listing 10: Makefile

```
.PHONY: all clean

all: report.txt

stats.txt: data.csv stats.py
    python stats.py

report.txt: report.md stats.txt build_report.py
    python build_report.py

clean:
    rm -f stats.txt report.txt
```

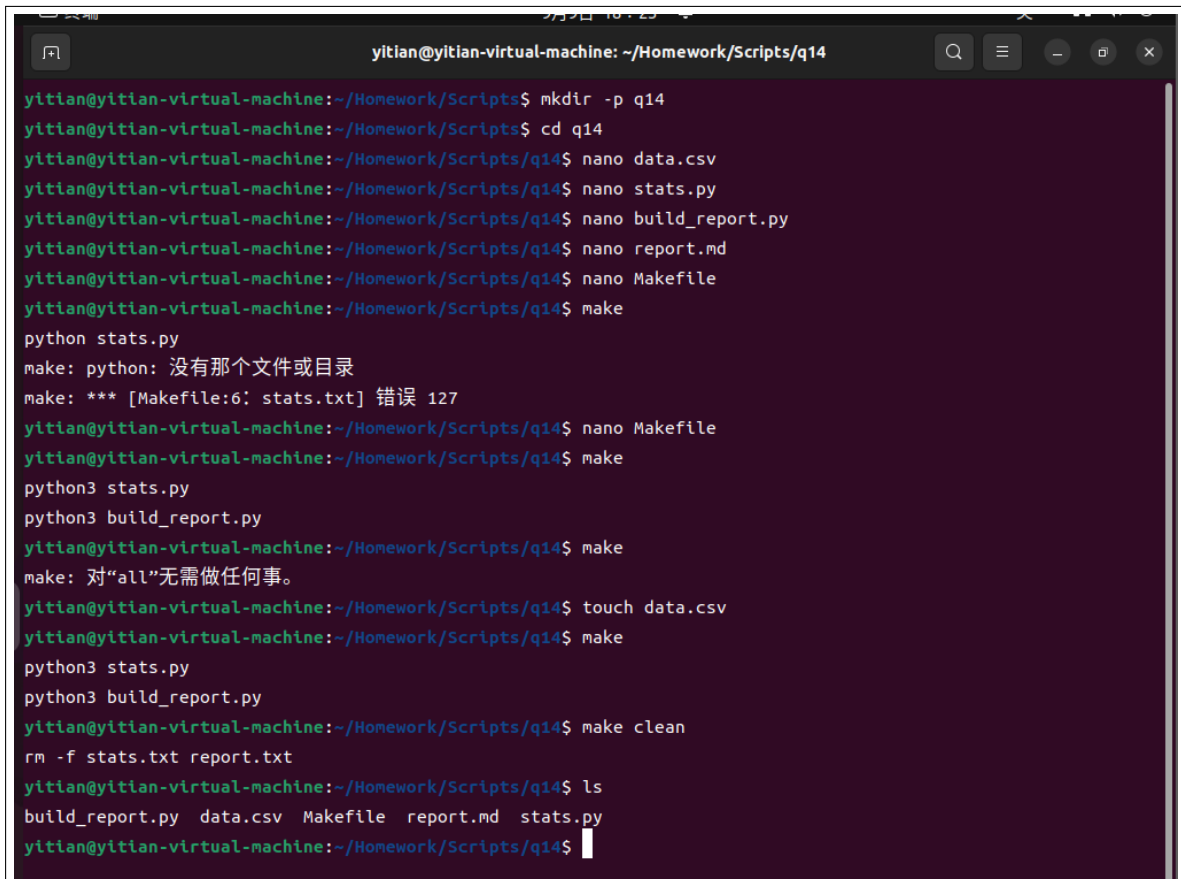
关键依赖关系:

- stats.txt 依赖 data.csv 和 stats.py
- report.txt 依赖 report.md、stats.txt 和 build_report.py
- all 和 clean 声明为.PHONY

3.2.3 3. 验证 Make 行为

1. 首次 make: 生成 stats.txt 和 report.txt
2. 第二次 make: 无改动, 不执行配方
3. touch data.csv 后 make: stats.txt 和 report.txt 重新生成
4. make clean: 只删除 stats.txt 和 report.txt

3.3 截图记录



```
yitian@yitian-virtual-machine: ~/Homework/Scripts/q14
yitian@yitian-virtual-machine:~/Homework/Scripts$ mkdir -p q14
yitian@yitian-virtual-machine:~/Homework/Scripts$ cd q14
yitian@yitian-virtual-machine:~/Homework/Scripts/q14$ nano data.csv
yitian@yitian-virtual-machine:~/Homework/Scripts/q14$ nano stats.py
yitian@yitian-virtual-machine:~/Homework/Scripts/q14$ nano build_report.py
yitian@yitian-virtual-machine:~/Homework/Scripts/q14$ nano report.md
yitian@yitian-virtual-machine:~/Homework/Scripts/q14$ nano Makefile
yitian@yitian-virtual-machine:~/Homework/Scripts/q14$ make
python stats.py
make: python: 没有那个文件或目录
make: *** [Makefile:6: stats.txt] 错误 127
yitian@yitian-virtual-machine:~/Homework/Scripts/q14$ nano Makefile
yitian@yitian-virtual-machine:~/Homework/Scripts/q14$ make
python3 stats.py
python3 build_report.py
yitian@yitian-virtual-machine:~/Homework/Scripts/q14$ make
make: 对“all”无需做任何事。
yitian@yitian-virtual-machine:~/Homework/Scripts/q14$ touch data.csv
yitian@yitian-virtual-machine:~/Homework/Scripts/q14$ make
python3 stats.py
python3 build_report.py
yitian@yitian-virtual-machine:~/Homework/Scripts/q14$ make clean
rm -f stats.txt report.txt
yitian@yitian-virtual-machine:~/Homework/Scripts/q14$ ls
build_report.py data.csv Makefile report.md stats.py
yitian@yitian-virtual-machine:~/Homework/Scripts/q14$
```

Figure 3: Q14 —Make 构建与依赖验证截图

3.4 GitHub Commit 记录

- **Commit 信息:**
feat(q14): add Makefile with correct dependency tracking
- **Commit 截图:** 见附录 6 (图 8)。

4 Q15: 把本地 API 数据转换为可读报告

4.1 题目要求

在 q15 中创建给定 JSON 和本地 HTTP 服务，再用 curl 与 jq 生成简短 Markdown 报告。

4.2 实现步骤

4.2.1 1. 创建 packages.json

Listing 11: packages.json

```
[
```

```
{ "name": "alpha", "status": "active", "downloads": 120, "version": "1.2.0" },
{ "name": "beta", "status": "inactive", "downloads": 900, "version": "2.0.0" },
{ "name": "gamma", "status": "active", "downloads": 450, "version": "1.5.1" },
{ "name": "delta", "status": "active", "downloads": 450, "version": "0.9.0" },
{ "name": "epsilon", "status": "active", "downloads": 80, "version": "3.1.0" }
]
```

4.2.2 2. 启动本地 HTTP 服务

```
python -m http.server 8000
```

4.2.3 3. 使用 curl 获取数据

```
curl -fsS http://127.0.0.1:8000/packages.json
```

4.2.4 4. 使用 jq 筛选与排序

筛选 status 为 active 且 downloads 不少于 100 的记录, 按 downloads 降序、name 升序排列:

Listing 12: jq 筛选命令

```
curl -fsS http://127.0.0.1:8000/packages.json \
| jq '[.[] | select(.status == "active" and .downloads >= 100)]
| sort_by(-.downloads, .name)'
```

筛选结果包含: gamma (450)、delta (450)、alpha (120)。

4.2.5 5. 编写 api_report.sh

Listing 13: api_report.sh

```
#!/usr/bin/env bash
set -euo pipefail

DATA=$(curl -fsS http://127.0.0.1:8000/packages.json)

echo "# Active Packages Report" > summary.md
echo "" >> summary.md
echo "| Name | Version | Downloads |" >> summary.md
echo "|-----|-----|-----|" >> summary.md

echo "$DATA" \
```

```
| jq -r '[.[] | select(.status == "active" and .downloads >= 100)]
| sort_by(-.downloads, .name)
| .[] | "| \(.name) | \(.version) | \(.downloads) |"' \
>> summary.md

echo "Report generated: summary.md"
```

生成的 summary.md 内容:

Listing 14: summary.md

```
# Active Packages Report

| Name      | Version | Downloads |
|-----|-----|-----|
| gamma     | 1.5.1   | 450       |
| delta     | 0.9.0   | 450       |
| alpha     | 1.2.0   | 120       |
```

4.3 截图记录

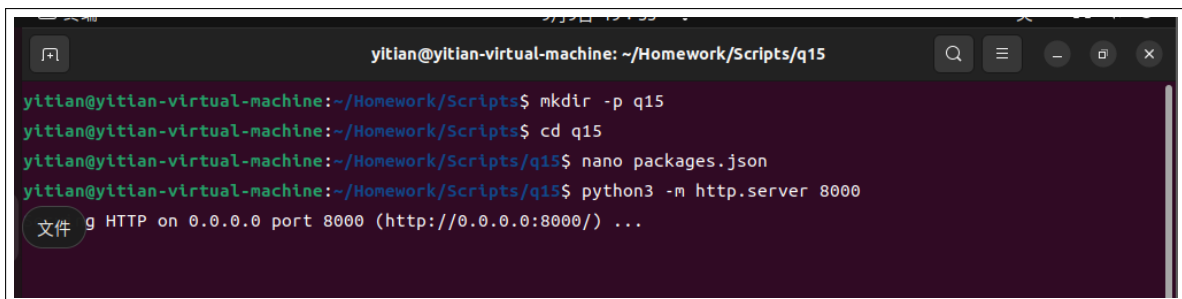


Figure 4: Q15 —HTTP 服务启动与 curl 获取数据截图

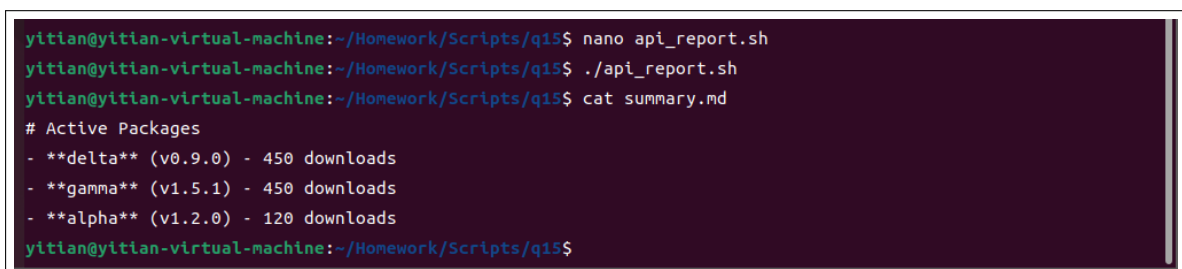


Figure 5: Q15 —jq 筛选与 summary.md 生成截图

4.4 GitHub Commit 记录

- **Commit 信息:**
feat(q15): generate Markdown report from local API with curl and jq
- **Commit 截图:** 见附录 6 (图 8)。

5 Q16: 修复并交付一个陌生的小型工具仓库

5.1 题目要求

复用第 13 题的项目，在 q16 中完成一次 15 分钟综合检查。

5.2 实现步骤

5.2.1 1. 复制项目并初始化 Git

将 q13 复制为 q16，初始化 Git 仓库：

```
cp -r q13 q16
cd q16
git init
git add .
git commit -m "chore(q16): initial commit from q13"
```

5.2.2 2. 临时修改导致测试失败

将问候语实现临时改成始终返回字面量 Hello, name!:

Listing 15: 临时修改—cli.py

```
import argparse

def main():
    p = argparse.ArgumentParser()
    p.add_argument("--name", required=True)
    a = p.parse_args()
    print("Hello, name!") # 临时修改: 始终返回字面量
```

运行 check.sh 确认测试失败——test_normal_name_greeting 期望 Hello, Alice! 但实际得到 Hello, name!。

5.2.3 3. 定位并修复

恢复正确的 f-string 实现：

Listing 16: 修复后—cli.py

```
import argparse

def main():
    p = argparse.ArgumentParser()
    p.add_argument("--name", required=True)
    a = p.parse_args()
    if not a.name.strip():
        raise SystemExit(2)
    print(f"Hello, {a.name}!")
```

再次运行 `check.sh`，全部测试通过。

5.2.4 4. 编写最小 Makefile

Listing 17: Makefile

```
.PHONY: check build clean

check:
    bash check.sh

build:
    python -m build

clean:
    rm -rf dist/ build/ *.egg-info
```

5.2.5 5. 构建与验证

```
make check    # 全部通过
make build    # 生成 wheel
```

计算 wheel 的 SHA-256:

Listing 18: SHA-256 计算

```
sha256sum dist/greetlab_25020007170-0.1.0-py3-none-any.whl
```

5.2.6 6. Git 提交

将最终改动提交为一个内容聚焦的 Git 提交:

```
fix(q16): restore correct greeting and add Makefile

- Reverted temporary literal "Hello, name!" change
- Added minimal Makefile with check/build/clean targets
- All checks pass, wheel built successfully
```

5.3 截图记录

5.4 GitHub Commit 记录

- **Commit 信息:**
fix(q16): restore correct greeting and add Makefile
- **Commit 截图:** 见附录 6 (图 8)。

```
yitian@yitian-virtual-machine:~/Homework/Scripts$ cd ~/Homework/Scripts/q13
yitian@yitian-virtual-machine:~/Homework/Scripts/q13$ mkdir -p .github/workflows
yitian@yitian-virtual-machine:~/Homework/Scripts/q13$ nano .github/workflows/ci.yml
yitian@yitian-virtual-machine:~/Homework/Scripts/q13$ ./check.sh
=== 1. Checking code format ===
3 files already formatted
=== 2. Checking code style ===
All checks passed!
=== 3. Running tests ===
===== test session starts =====
platform linux -- Python 3.10.12, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/yitian/Homework/Scripts/q13
configfile: pyproject.toml
testpaths: tests
collected 3 items

tests/test_cli.py ... [100%]

===== 3 passed in 0.05s =====
✓ All checks passed!
yitian@yitian-virtual-machine:~/Homework/Scripts/q13$ git add .github workflows/check.sh pyproject.toml src tests
fatal: 不是 git 仓库 (或者任何父目录) : .git
yitian@yitian-virtual-machine:~/Homework/Scripts/q13$ git init
提示: 使用 'master' 作为初始分支的名称。这个默认分支名称可能会更改。要在新仓库中
提示: 配置使用初始分支名, 并消除这条警告, 请执行:
提示:
提示: git config --global init.defaultBranch <名称>
提示:
提示: 除了 'master' 之外, 通常选定的名字有 'main'、'trunk' 和 'development'。
提示: 可以通过以下命令重命名刚创建的分支:
提示:
提示: git branch -m <name>
已初始化空的 Git 仓库于 /home/yitian/Homework/Scripts/q13/.git/
yitian@yitian-virtual-machine:~/Homework/Scripts/q13$ git branch -M main
yitian@yitian-virtual-machine:~/Homework/Scripts/q13$ git add .
yitian@yitian-virtual-machine:~/Homework/Scripts/q13$ git commit -m "Add q14 CI pipeline + check script"
```

Figure 6: Q16 一测试失败与修复过程截图

6 实验总结

—报告结束—

```
yitian@yitian-virtual-machine:~/Homework/Scripts$ cp -r q13 q16
yitian@yitian-virtual-machine:~/Homework/Scripts$ cd q16
yitian@yitian-virtual-machine:~/Homework/Scripts/q16$ git init
已重新初始化已存在的 Git 仓库于 /home/yitian/Homework/Scripts/q16/.git/
yitian@yitian-virtual-machine:~/Homework/Scripts/q16$ git branch -M main
yitian@yitian-virtual-machine:~/Homework/Scripts/q16$ git add .
yitian@yitian-virtual-machine:~/Homework/Scripts/q16$ git commit -m "Initial commit from q13"
Rhythmbox in
无文件要提交，干净的工作区
yitian@yitian-virtual-machine:~/Homework/Scripts/q16$ nano src/greetlab/cli.py
yitian@yitian-virtual-machine:~/Homework/Scripts/q16$ ./check.sh
=== 1. Checking code format ===
3 files already formatted
=== 2. Checking code style ===
All checks passed!
=== 3. Running tests ===
===== test session starts =====
platform linux -- Python 3.10.12, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/yitian/Homework/Scripts/q16
configfile: pyproject.toml
testpaths: tests
collected 3 items

tests/test_cli.py ... [100%]

===== 3 passed in 0.06s =====
✓ All checks passed!
```

Figure 7: Q16 —make check / make build 与 SHA-256 截图

附录 A: Git 提交记录

仓库地址: <https://github.com/ADayxs/SystemDevelopmentTools.git>

本次实验共 4 次提交，均已写入个人 GitHub 仓库（遵守“禁止单次全量提交”要求，按题目分次提交）。提交记录截图如下：

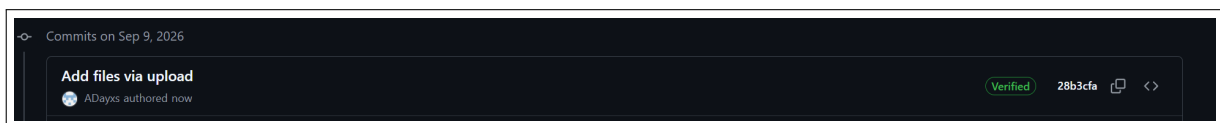


Figure 8: GitHub Commit 记录截图（含 Q13–Q16 各次提交）

题号	收获与反思	完成度
Q13	掌握了 ruff 格式化与静态检查的配置方法，理解了本地质量门禁在 CI 之前的重要性	✓
Q14	深入理解了 Make 的依赖追踪机制，学会通过正确的依赖关系避免不必要的重建	✓
Q15	综合运用 curl、jq 和 Shell 脚本，实现了从 API 数据到 Markdown 报告的自动化流程	✓
Q16	在限时条件下完成了项目修复、构建和提交的完整流程，提升了综合调试能力	✓